



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lab 5 — Capstone — Test & Deliver the Lecture 4 API

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lab 5 of 5
Focus	Start from the completed Lecture 4 SQLite ORM project, verify its Alembic revision, and demonstrate the same APIRouter CRUD endpoints using Swagger, curl and Postman.
Evaluation	Lab Assignment — 100%

Before You Start

This lab continues directly from Lecture 4. Keep the same documents.db, migrated description column, APIRouter structure, Pydantic schemas, and direct with Session(engine) as db pattern. Do not rebuild the database or switch to Depends.

Today's Lab Plan

1. Verify the Lecture 4 project and migration state
2. Test the full CRUD lifecycle with curl
3. Create a Postman collection and environment
4. Record success and failure contract results
5. Perform the final integration review
6. Prepare the lab assignment submission

01

Verify Lecture 4 & Run the API

Goal: Prove that the Lecture 4 project is on the expected Alembic revision and starts using the same modular API structure.

Steps

1. Open the completed Lecture 4 project.
2. Confirm alembic.ini points to sqlite:///documents.db.
3. Run `python -m alembic current`; use `upgrade head` if the database is behind.
4. Start Uvicorn with `uvicorn main:app --reload`.
5. Verify `/docs`, `/redoc` and `/openapi.json` open successfully.

Starter / reference code

```
python -m alembic current
# if needed:
python -m alembic upgrade head

uvicorn main:app --reload

# Open in browser
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/redoc
```

✓ CHECKPOINT — Exercise 1

Q1. Why check the migration revision before final testing?

Answer: ??

02

curl CRUD Smoke Test — Windows CMD

Goal: Exercise the Lecture 4 document lifecycle from an external command-line HTTP client.

Steps

1. GET `/documents` and confirm 200.
2. POST one valid document with description and note the returned id.
3. GET the new document and confirm 200.
4. PUT an update and confirm the changed values.
5. DELETE the document and confirm 200.
6. GET it again and confirm 404.
7. Send one invalid POST and confirm 422.

Starter / reference code

```
curl -i http://127.0.0.1:8000/documents

curl -i -X POST http://127.0.0.1:8000/documents ^
-H "Content-Type: application/json" ^
-d "{\"title\":\"Curl Doc\",\"content\":\"capstone\",\"priority\":3,\"description\":\"Lecture 5\"}"

curl -i http://127.0.0.1:8000/documents/1

curl -i -X DELETE http://127.0.0.1:8000/documents/1
curl -i http://127.0.0.1:8000/documents/1
```

✓ CHECKPOINT — Exercise 2

Q1. What should every curl check record?

Answer: ??

03 Postman Collection & Environment

Goal: Build a reusable visual test suite for the exact Lecture 4 API.

Steps

1. Create base_url = http://127.0.0.1:8000 in a Postman environment.
2. Create list, create, get one, update and delete requests for /documents.
3. Add one invalid create request (422) and one missing-id request (404).
4. Use document_id in {{base_url}}/documents/{{document_id}} and save clear request names.
5. Export the Postman collection JSON for submission.

✓ CHECKPOINT — Exercise 3

Q1. Why use {{base_url}}?

Answer: ??

04 Contract Verification Matrix

Goal: Compare expected and actual HTTP behavior for the current migrated API.

Steps

1. Record method/path, test case, expected code, actual code and pass/fail.
2. Include 200, 201, 404 and 422 cases.
3. Confirm response bodies match DocumentResponse where applicable.
4. Confirm missing resources return 404 instead of an internal 500 error.

✓ CHECKPOINT — Exercise 4

Q1. Is a 200 response enough to call a test passed?

Answer: ??

Method / Path	Case	Expected
GET /documents	normal	200
POST /documents	valid body	201
POST /documents	invalid body	422
GET /documents/999999	missing	404
DELETE /documents/{id}	existing	200

05

Final Production-Readiness Review

Goal: Confirm that the final submission is a tested continuation of Lecture 4, not a different architecture.

Steps

1. `main.py` includes the Documents APIRouter.
2. `models.py` contains the migrated optional description column.
3. Pydantic request/response schemas include description.
4. Database endpoints still use with `Session(engine)` as db.
5. Alembic current/upgrade head works for `documents.db`.
6. Missing documents return 404 and invalid bodies return 422.
7. `/docs` and `/redoc` match the real endpoint behavior.
8. Swagger, curl and Postman results are reproducible.

✓ CHECKPOINT — Exercise 5

Q1. What architecture should the final lab use?

Answer: ??

06

Lab Assignment Submission — 100%

- Source code: `main.py`, `database.py`, `models.py`, `schemas.py`, and `routes/`.
- `alembic.ini` plus the `alembic/` migration directory.
- `requirements.txt` or a short dependency list.
- Exported Postman collection JSON.
- A short README with migrate, run, Swagger, curl and Postman test steps.
- Evidence of curl tests for success, 404 and 422 cases.
- Completed contract verification matrix.